Project Report

Yulin Wu, #21130792

November 15, 2024

1 Introduction

The vertex cover problem is an NP-hard problem that tries to find a subset of the vertices in a graph, such that every edge in the graph is incident on some vertex in the subset. This subset is called a vertex cover. The objective of minimum vertex cover problem is to find the minimum size of a vertex cover.

In the previous assignment of the course ECE 650 in 2024 Fall, an approach that encodes the problem into conjunctive normal form (CNF) and use a SAT solver to compute the minimum vertex cover. This project implements two approximation method to compute the minimum vertex cover and compares the measure of running time and approximation ratio of different methods based on experiments of different graphs. The major find of the result shows that the CNF methods runs really slow when the graph has bigger size and approximation method can yield great approximation ratio. This gives us an idea that sometimes we can use approximation algorithm to trade time efficiency with precision. In the future development, some heuristic approximation methods or better CNF encoding can be use to improve the performance of solving this problem.

This project report is a summary and analysis of the course project of ECE 650 in 2024 Fall. In the following sections, the report first discusses the program structure and test dataset. Then, the report shows the visualized results of running time and approximation ratio, and gives some explanation of them. Finally, the report provides a conclusion and gives some advice for future development.

2 Program Structure and Test Dataset

The program of this project consists of four threads: one main thread and three other threads that compute the minimum vertex cover problems in different approaches. The main thread deals with input graph parsing and output results, creates threads to solve the problem and controls other threads.

In addition, The project implements a data structure, Graph, to store parsed graph information, i.e., edges and vertices, and also adds functions to the data structure that compute the minimum vertex cover problems in different approaches, namely CNF-SAT-VC, APPROX-1-VC and APPROX-2-VC. After the main thread parses the graph and creates a Graph data instance, it will create three solver threads, each of which calls different solving functions in the instance and collects analytic data for visualizing the running time. The main thread will wait for a reasonable amount of time for the solver threads to process, and after the time expires, try to cancel the threads that are still running and format the vertex cover result or analytic data for output printing, based on the mode macro defined in the file.

This project chose the timeout as $1,600$ milliseconds. This timeout should not be too large as the project needs to run each graph ten times; each $|V|$ has ten graphs; and it is required to collect data for ten different $|V|$. The total run would be $1,000$ times. Besides, based on the previous development on assignment 4, the CNF-SAT-VC approach runs exponentially slower as the number of vertices increases. When the approach tries vertex cover size of 7, the running time would be no more than $1,000$ milliseconds, which is a reasonable timeout. But when trying the size of 8, the running time would be a few minutes, which is too much for the timeout. Therefore, considering other time cost of other routine such as I/O, the timeout is chosen as $1,600$ milliseconds.

The test dataset used in this project is generated by a given application "GraphGen", the implementation detail of which is not given. This application generates graphs based on the number of vertices. Based on observation, it creates random graphs with the number of edges equal to the number of vertices and also guaranteed that every vertex belongs to some edge.

The test dataset is created in accordance to the project requirement: ten graphs for each $|V|$ and $|V|$ ranges from 5 to 50 in increments of 5.

3 Result and Discussion

3.1 Running Time

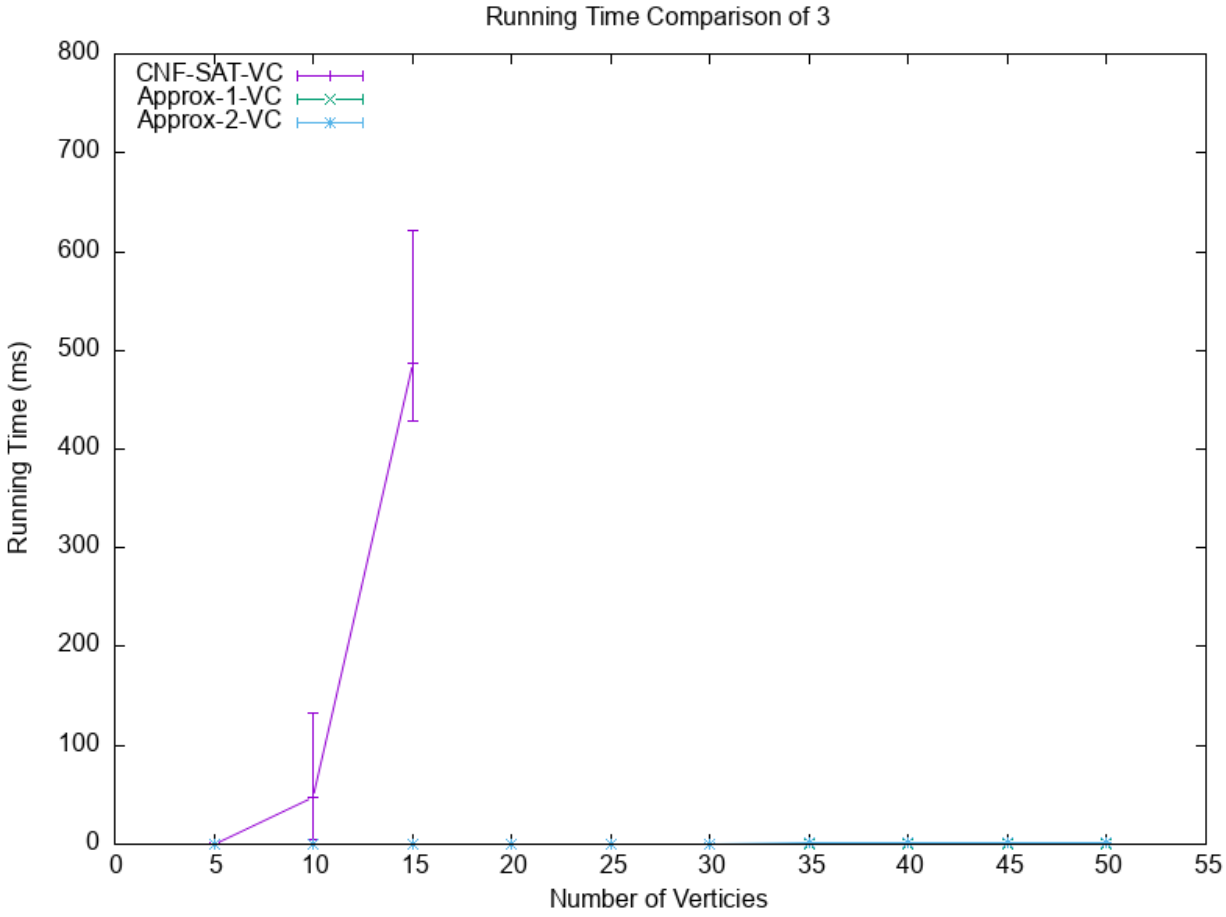

Figure 1: Running Time Comparison of CNF-SAT-VC, Approx-1-VC and Approx-2-VC

The running time of this project is measured by milliseconds. Figure 1 is the running time comparison of all three approaches. Figure 2 is a zoomed in running time comparison of Approx-1-VC and Approx-2-VC.

As shown in Figure 1, the time consumed by CNF-SAT-VC is significantly larger than the other two approaches as the number of vertices increases. The trend of the running time of CNF-SAT-VC as the number of vertices increases is exponential increase trend. The reason for this trend is that the CNF-SAT-VC approach enumerates the size of vertex cover in ascending order. Before finding the minimum vertex cover size, the approach already tried smaller size that will cause the encoded CNF unsatisfied. Since proving a CNF is unsatisfied runs at exponential time, the running time of the CNF-SAT-VC approach is also exponential. Eventually, when the number of vertices is greater and equal than 20, all graphs using CNF-SAT-VC approach result in timeout and this report omits the timeout result from the plot as timeout running time results are meaningless.

The running time standard deviation of CNF-SAT-VC is relatively large. This unstable performance might result from the allocation routine of the CNF-SAT-VC approach. That is the allocation of variables and the SAT solver, and the solver itself might use some allocation when running. These allocation operations touch the computer memory, and the running time is impacted by the operating system and the number of users sharing the computer, leading to unstable performance.

Figure 2 gives a running time comparison of Approx-1-VC and Approx-2-VC.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

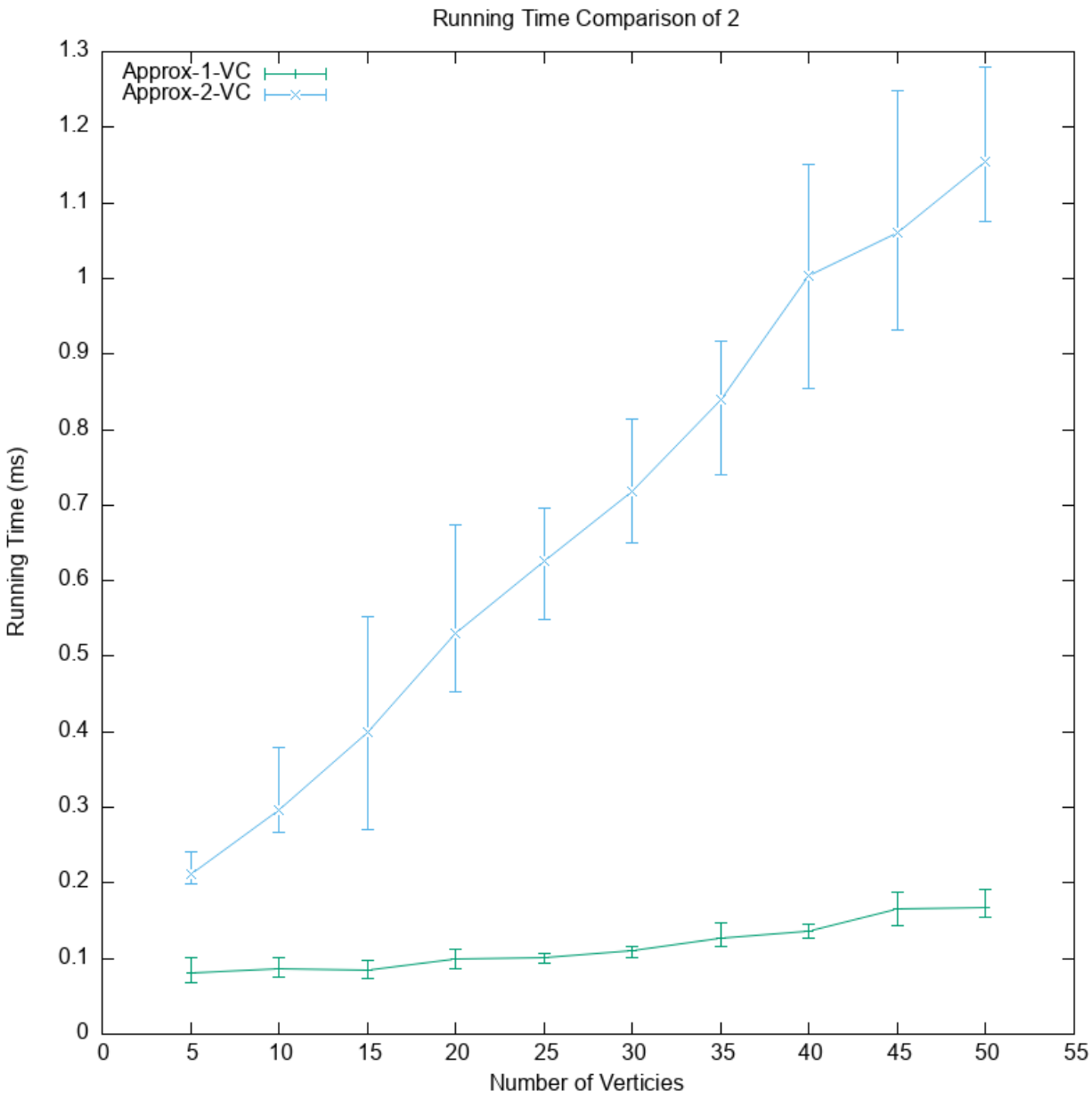

Figure 2: Running Time Comparison of Approx-1-VC and Approx-2-VC

3.2 Effective Ratio

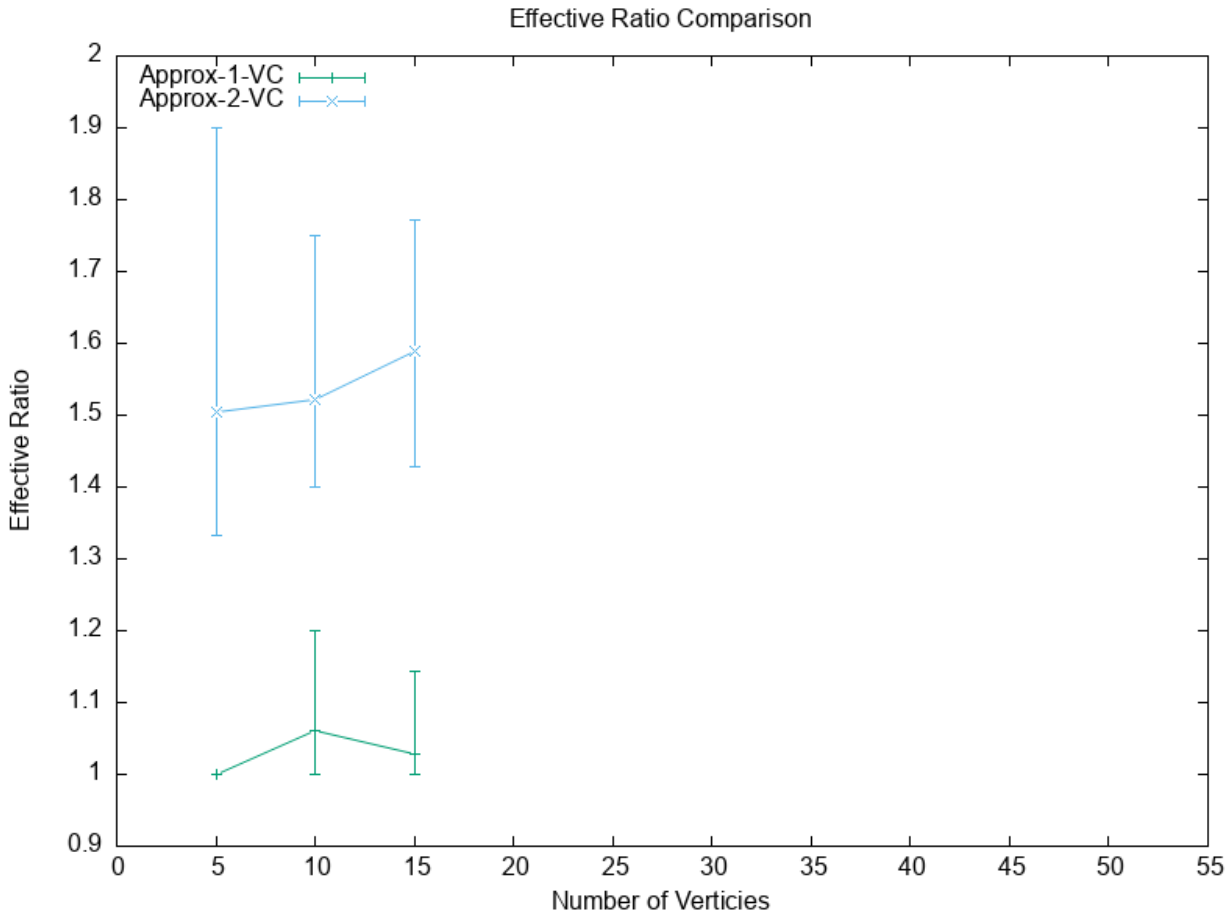

Figure 3: Effective Ratio Comparison of Approx-1-VC and Approx-2-VC

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et

neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

4 Conclusion

The major find of the result shows that the CNF methods runs really slow when the graph has bigger size and approximation method can yield great approximation ratio.